

DBMS Practical Examinations - SET 3

E-Commerce Platform Scenario SQL Codes & Execution Outputs

Prerequisite: Database Schema Creation

```
CREATE TABLE Customers (  
    customer_id INT PRIMARY KEY AUTO_INCREMENT,  
    name VARCHAR(100) NOT NULL,  
    email VARCHAR(100) UNIQUE NOT NULL,  
    city VARCHAR(50)  
);
```

```
CREATE TABLE Products (  
    product_id INT PRIMARY KEY AUTO_INCREMENT,  
    product_name VARCHAR(100) NOT NULL,  
    price DECIMAL(10,2) NOT NULL,  
    stock_qty INT NOT NULL  
);
```

```
CREATE TABLE Orders (  
    order_id INT PRIMARY KEY AUTO_INCREMENT,  
    customer_id INT,  
    order_date DATE,  
    total_amount DECIMAL(10,2),  
    FOREIGN KEY (customer_id) REFERENCES Customers(customer_id) ON DELETE  
CASCADE  
);
```

```
CREATE TABLE OrderItems (  
    item_id INT PRIMARY KEY AUTO_INCREMENT,  
    order_id INT,  
    product_id INT,  
    quantity INT NOT NULL,  
    subtotal DECIMAL(10,2) NOT NULL,  
    FOREIGN KEY (order_id) REFERENCES Orders(order_id) ON DELETE  
CASCADE,  
    FOREIGN KEY (product_id) REFERENCES Products(product_id) ON DELETE  
CASCADE  
);
```

Output / Execution Result:

```
mysql>
mysql> CREATE TABLE Orders (
  ->   order_id INT PRIMARY KEY AUTO_INCREMENT,
  ->   customer_id INT,
  ->   order_date DATE,
  ->   total_amount DECIMAL(10,2),
  ->   FOREIGN KEY (customer_id) REFERENCES Customers(customer_id) ON DELETE CASCADE
  -> );
Query OK, 0 rows affected (0.06 sec)

mysql>
mysql> CREATE TABLE OrderItems (
  ->   item_id INT PRIMARY KEY AUTO_INCREMENT,
  ->   order_id INT,
  ->   product_id INT,
  ->   quantity INT NOT NULL,
  ->   subtotal DECIMAL(10,2) NOT NULL,
  ->   FOREIGN KEY (order_id) REFERENCES Orders(order_id) ON DELETE CASCADE,
  ->   FOREIGN KEY (product_id) REFERENCES Products(product_id) ON DELETE CASCADE
  -> );
Query OK, 0 rows affected (0.06 sec)
```

1. SQL & Joins

SELECT

 c.name,
 COUNT(o.order_id) AS total_orders,
 SUM(o.total_amount) AS total_amount_spent

FROM Customers c

INNER JOIN Orders o ON c.customer_id = o.customer_id

GROUP BY c.customer_id, c.name

ORDER BY total_amount_spent DESC;

Output / Execution Result:

```
mysql>
mysql> -- 4. Insert sample OrderItems
mysql> INSERT INTO OrderItems (order_id, product_id, quantity, subtotal) VALUES
    -> (1, 1, 1, 1200.00),
    -> (1, 3, 1, 150.00),
    -> (2, 2, 1, 800.00),
    -> (3, 3, 1, 150.00);
Query OK, 4 rows affected (0.01 sec)
Records: 4  Duplicates: 0  Warnings: 0

mysql> SELECT
    -> c.name,
    -> COUNT(o.order_id) AS total_orders,
    -> SUM(o.total_amount) AS total_amount_spent
    -> FROM Customers c
    -> INNER JOIN Orders o ON c.customer_id = o.customer_id
    -> GROUP BY c.customer_id, c.name
    -> ORDER BY total_amount_spent DESC;
+-----+-----+-----+
| name      | total_orders | total_amount_spent |
+-----+-----+-----+
| Alice Smith | 2           | 1500.00            |
| Bob Jones  | 1           | 800.00              |
+-----+-----+-----+
2 rows in set (0.01 sec)
```

2. Views

-- Create the View

```
CREATE VIEW vw_high_value_orders AS
```

```
SELECT
```

```
    o.order_id,
    c.name AS customer_name,
    o.order_date,
    o.total_amount
```

```
FROM Orders o
```

```
INNER JOIN Customers c ON o.customer_id = c.customer_id
```

```
WHERE o.total_amount > 1000;
```

-- Retrieve all rows from the view

```
SELECT * FROM vw_high_value_orders;
```

Output / Execution Result:

```
-> ORDER BY total_amount_spent DESC;
+-----+-----+-----+
| name      | total_orders | total_amount_spent |
+-----+-----+-----+
| Alice Smith |          2 |          1500.00 |
| Bob Jones  |          1 |           800.00 |
+-----+-----+-----+
2 rows in set (0.01 sec)

mysql> CREATE VIEW vw_high_value_orders AS
-> SELECT
->     o.order_id,
->     c.name AS customer_name,
->     o.order_date,
->     o.total_amount
-> FROM Orders o
-> INNER JOIN Customers c ON o.customer_id = c.customer_id
-> WHERE o.total_amount > 1000;
ERROR 1050 (42S01): Table 'vw_high_value_orders' already exists
mysql>
mysql> -- Retrieve all rows from the view
mysql> SELECT * FROM vw_high_value_orders;
+-----+-----+-----+-----+
| order_id | customer_name | order_date | total_amount |
+-----+-----+-----+-----+
|         1 | Alice Smith   | 2023-10-01 |          1350.00 |
+-----+-----+-----+-----+
```

3. User-Defined Function

DELIMITER //

```
CREATE FUNCTION fn_get_discounted_price(
    p_product_id INT,
    p_discount_percentage DECIMAL(5,2)
)
RETURNS DECIMAL(10,2)
READS SQL DATA
BEGIN
    DECLARE v_base_price DECIMAL(10,2);
    DECLARE v_final_price DECIMAL(10,2);

    -- Retrieve the current price of the product
    SELECT price INTO v_base_price
    FROM Products
    WHERE product_id = p_product_id;

    -- Calculate final price (price - discount amount)
    SET v_final_price = v_base_price - (v_base_price * (p_discount_percentage / 100));

    RETURN v_final_price;
END //
```

DELIMITER ;

Output / Execution Result:

```
mysql> DELIMITER //
mysql>
mysql> CREATE FUNCTION fn_get_discounted_price(
  ->     p_product_id INT,
  ->     p_discount_percentage DECIMAL(5,2)
  -> )
  -> RETURNS DECIMAL(10,2)
  -> READS SQL DATA
  -> BEGIN
  ->     DECLARE v_base_price DECIMAL(10,2);
  ->     DECLARE v_final_price DECIMAL(10,2);
  ->
  ->     -- Retrieve the current price of the product
  ->     SELECT price INTO v_base_price
  ->     FROM Products
  ->     WHERE product_id = p_product_id;
  ->
  ->     -- Calculate final price (price - discount amount)
  ->     SET v_final_price = v_base_price - (v_base_price * (p_discount_percentage / 100));
  ->
  ->     RETURN v_final_price;
  -> END //
Query OK, 0 rows affected (0.09 sec)
```

4. Triggers

DELIMITER //

```
CREATE TRIGGER trg_update_stock
AFTER INSERT ON OrderItems
FOR EACH ROW
BEGIN
    UPDATE Products
    SET stock_qty = stock_qty - NEW.quantity
    WHERE product_id = NEW.product_id;
END //
```

DELIMITER

Output / Execution Result:

```
mysql> CREATE TRIGGER trg_update_stock
  -> AFTER INSERT ON OrderItems
  -> FOR EACH ROW
  -> BEGIN
  ->     UPDATE Products
  ->     SET stock_qty = stock_qty - NEW.quantity
  ->     WHERE product_id = NEW.product_id;
  -> END //
Query OK, 0 rows affected (0.04 sec)
```